UỶ BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP NHÓM TUẦN

Lab 03 - Local search

Sinh viên thực hiện: Trương Quốc Lê Huy 3123410132
Hồ Hoàng Khang 3123410150
Tô Gia Huy 3123410131

Học phần: Trí tuệ nhân tạo nâng cao

Giáo viên hướng dẫn Đỗ Như Tài

Thành phố Hồ Chí Minh, Tháng 10 Năm 2026

Bảng phân công công việc

STT	Họ và tên	MSSV	Phân Công	Mức Độ Hoàn Thành
1	Tô Gia Huy	3123410131	Bài 4, 5, 6	33,3%
2	Trương Quốc Lê Huy	3123410132	Bài 7,8 và tổng hợp báo cáo	33,4%
3	Hồ Hoàng Khang	3123410150	Bài 1, 2, 3	33,3%

MỤC LỤC

Bảng phân công công việc **2**
MỤC LỤC **3**
TÓM TẮT **4**
CHƯƠNG 1 TÌM KIẾM CỤC BỘ TRÊN BÀI TOÁN MÊ CUNG (MAZE) **5**
1.1 Kết quả từ result["solution"], vị trí dừng và so sánh với đích G 5
1.2 Giải thích lý do thuật toán dừng dù chưa đến đích 5
1.3 Nhận xét khi thay đổi thứ tự xét hướng (ORDER) hoặc ngẫu nhiên (RANDOMIZATION) 6
1.4 Kết luận tổng quan 6
CHƯƠNG 2 BÀI TOÁN N-QUEENS VÀ CÁC BIẾN THỂ HILL CLIMBING **8**
2.1 Tìm kiếm leo đồi dốc nhất và leo đồi ngẫu nhiên 8
2.1.1. Khảo sát các hàm bổ trợ trong n_queens.ipynb 8
2.1.2. Biểu diễn trạng thái của bài toán 8
2.1.3. Phép di chuyển cục bộ (Local Move) và số trạng thái láng giềng 8
2.1.4. Thực hiện thử nghiệm và so sánh cấu hình bàn cờ (n = 4 và n = 8) 9
2.2 Random Restart 9
2.2.1. Biểu diễn trạng thái và hàm đánh giá 10
2.2.2. Random Restart Hill Climbing 10
2.2.3. Ba biến thể Hill Climbing 10
2.2.4. Kết quả thực nghiệm 11
2.3 So sánh các lịch giảm nhiệt trong Simulated Annealing 12
2.3.1. Thiết lập thí nghiệm 12
2.3.2. Kết quả thực nghiệm 12
CHƯƠNG 3 TỐI ƯU HÓA TÍNH TOÁN VỚI NUMBA **16**
3.1 Cơ sở lý thuyết 16
3.2 Ba phương pháp cài đặt 16
3.2.1. conflicts_naive() — $O(n^2)$ 16
3.2.2. conflicts() — O(n) 16
3.2.3. conflicts_jit() — Numba JIT, O(n) 16
3.3 Kiểm tra tính đúng đắn 17
3.4 Thiết lập đo thời gian 17
3.5 Kết quả benchmark 17
3.6 Đồ thị và nhận xét 18
3.7 Phân tích lợi ích của Numba JIT 19
3.8 Kết luận 19
CHƯƠNG 4 BÀI TOÁN NGƯỜI CHÀO HÀNG (TSP) VÀ THUẬT TOÁN DI TRUYỀN (GA) **20**
3.1 Mô hình hóa bài toán và Toán tử lân cận 20
3.2 Thực nghiệm với các thuật toán Tìm kiếm cục bộ 20

3.3 Ứng dụng Thuật toán Di truyền (Genetic Algorithm)..... 21
CHƯƠNG 5 ĐÁNH GIÁ SỰ HỘI TỤ, KHẢ NĂNG MỞ RỘNG VÀ TÍNH ĐÁNH ĐỔI.....23
5.1 Bảng thống kê kết quả thực nghiệm..... 23
5.2. Mô hình hội tụ và Khả năng mở rộng (Scalability).....23
5.3. Nhận xét về sự đánh đổi (Trade-off)..... 24
CHƯƠNG 6 TRẢ LỜI CÂU HỎI THẢO LUẬN CHUNG.....26
TÀI LIỆU THAM KHẢO..... 28

TÓM TẮT

Qua quá trình thực nghiệm, nhóm đã triển khai và phân tích thành công các thuật toán Tìm kiếm cục bộ từ cơ bản đến nâng cao trên ba bài toán đặc trưng: Tìm đường trong mê cung, xếp hậu và người bán hàng.

Kết quả thực nghiệm đã làm rõ những đặc tính cốt lõi và sự đánh đổi của từng phương pháp:

- Thuật toán Leo đồi: Thể hiện ưu điểm về tốc độ cải thiện chất lượng lời giải ở giai đoạn đầu, nhưng bộc lộ rõ hạn chế dễ mắc kẹt tại các cực trị cục bộ do bản chất tham lam. Biến thể lựa chọn đầu tiên là một giải pháp xuất sắc giúp cân bằng giữa thời gian tính toán và hướng đi lên đỉnh đồi.
- Thoát khỏi bế tắc: Các chiến lược như Khởi tạo lại ngẫu nhiên và Luyện kim mô phỏng đã chứng minh được hiệu quả trong việc vượt qua các thung lũng cục bộ. Đặc biệt, Luyện kim mô phỏng cho thấy khả năng hội tụ về nghiệm tối ưu toàn cục rất trơn tru nếu được thiết lập lịch giảm nhiệt độ phù hợp.
- Tối ưu hóa và mở rộng: Việc kết hợp tối ưu mã nguồn bằng công cụ Numba và cấu trúc dữ liệu mảng một chiều đã mang lại tốc độ thực thi vượt trội, minh chứng cho tầm quan trọng của kỹ năng lập trình bên cạnh tư duy thuật toán. Ngoài ra, việc áp dụng Thuật toán Di truyền đã củng cố thêm phương pháp giải quyết các không gian trạng thái khổng lồ bằng cách tiếp cận dựa trên quần thể sinh học.

CHƯƠNG 1 TÌM KIẾM CỤC BỘ TRÊN BÀI TOÁN MÊ CUNG (MAZE)

Thuật toán tìm kiếm cục bộ bắt đầu từ một trạng thái hiện tại và liên tục di chuyển sang các trạng thái lân cận tốt hơn. Bài toán mê cung minh họa rõ nét cách tác tử định hướng di chuyển thông qua việc đánh giá khoảng cách Manhattan đến đích.

1.1 Kết quả từ result["solution"], vị trí dừng và so sánh với đích G

Vị trí xuất phát (S): Tọa độ (9, 5).

Vị trí đích (G): Tọa độ (2, 12).

Vị trí dừng (result["solution"]): Ô [5, 9].

Khoảng cách Manhattan h(n) khi dừng: Tại ô [5, 9], giá trị hàm mục tiêu thu được là h(n) = 6.

So sánh với đích G:

- Ô dừng [5, 9] vẫn còn cách đích G(2, 12) một khoảng cách Manhattan là |5 - 2| + |9 - 12| = 3 + 3 = 6.
- Thuật toán xuất thông báo "No solution found." và dừng lại giữa đường, hoàn toàn chưa tìm được đường tới đích G.

1.2 Giải thích lý do thuật toán dừng dù chưa đến đích

Chiến lược hoạt động của Simple Hill Climbing:

- Thuật toán tìm kiếm theo chiến lược cục bộ tham lam (Greedy Local Search).
- Tại vị trí hiện tại current, thuật toán gọi hàm expand() để tìm tất cả các ô lân cận hợp lệ (không phải bức tường 'X').
- Tính khoảng cách Manhattan h(n) từ mỗi ô lân cận tới đích G.
- Chọn ô lân cận có giá trị h(n) nhỏ nhất (best_obj).
- Điều kiện chuyển trạng thái: Chỉ di chuyển sang ô mới nếu best_obj < current_obj (khoảng cách tới đích thực sự được thu ngắn). Nếu không có ô lân cận nào tốt hơn, thuật toán sẽ thực hiện lệnh break để dừng lại.

Nguyên nhân cụ thể khiến thuật toán bị kẹt:

- Khi di chuyển tới ô [5, 9], giá trị hàm mục tiêu đạt h(n) = 6.
- Hai ô lân cận đi tiếp hợp lệ duy nhất từ ô [5, 9] là [6, 9] và [5, 8].
- Cả hai ô lân cận này đều có khoảng cách Manhattan tới G(2, 12) tăng lên thành 7 (h(n) = 7).

- Do 7 > 6, không có ô lân cận nào thỏa mãn điều kiện best_obj < current_obj.
- Thuật toán bị rơi vào Cực trị cục bộ (Local Optimum / Dead End). Vì Hill Climbing không có cơ chế quay xe (backtracking) hay chấp nhận chuyển sang vị trí có chi phí xấu hơn, nó buộc phải dừng lại dù chưa tới đích.

1.3 Nhận xét khi thay đổi thứ tự xét hướng (ORDER) hoặc ngẫu nhiên (RANDOMIZATION)

Khi thay đổi thứ tự xét hướng cố định (Ví dụ: ORDER = "SWNE", "ESWN",...):

- Kết quả: Thuật toán vẫn bị mắc kẹt ở các vị trí ngõ cụt và không tìm thấy đường tới đích G.
- Nhận xét: Thay đổi thứ tự hướng chỉ làm thay đổi lựa chọn giữa các ô có cùng giá trị h(n). Với cấu trúc tường hình chữ L của L_maze.txt, hàm khoảng cách Manhattan thuần túy luôn kéo thuật toán đi về phía bên phải trước khi chạm tường, dẫn đến việc luôn bị kẹt ở góc tường.

Khi chọn hướng ngẫu nhiên (RANDOMIZATION = True):

- Quá trình chạy: Đường đi của thuật toán qua từng bước lặp sẽ thay đổi ngẫu nhiên giữa các lần chạy khác nhau.
- Kết quả: Đa số các lần chạy vẫn bị mắc kẹt tại ô [5, 9] hoặc các ô góc tường lân cận.
- Nhận xét: Yếu tố ngẫu nhiên làm tăng tính đa dạng của hành trình nhưng không khắc phục được hạn chế cốt lõi của Simple Hill Climbing. Thuật toán hoàn toàn thiếu bộ nhớ lưu vết (không lưu danh sách ô đã đi visited) và không có cơ chế chấp nhận rủi ro (như các thuật toán Simulated Annealing hay Random Restart Hill Climbing) để thoát khỏi cực trị cục bộ.

1.4 Kết luận tổng quan

Ưu điểm:

- Cấu trúc đơn giản, dễ cài đặt và triển khai.
- Tốc độ xử lý nhanh, tiết kiệm bộ nhớ do không cần lưu trữ cây tìm kiếm hay tập mở/đóng.

Nhược điểm:

- Rất dễ bị kẹt tại các vị trí Cực trị cục bộ (Local Optima), Vùng bằng phẳng (Plateau) hoặc Dông núi (Ridge).

- Không đảm bảo tính Hoàn chỉnh (Completeness) — có thể báo không tìm thấy đường dù lời giải thực sự tồn tại

CHƯƠNG 2 BÀI TOÁN N-QUEENS VÀ CÁC BIẾN THỂ HILL CLIMBING

Bài toán N-Queens yêu cầu đặt N quân hậu lên bàn cờ N×N sao cho không có quân nào tấn công nhau. Hàm đánh giá là số lượng cặp hậu xung đột.

2.1 Tìm kiếm leo đồi dốc nhất và leo đồi ngẫu nhiên

2.1.1. Khảo sát các hàm bổ trợ trong n_queens.ipynb

Hàm random_board(n):

- Tác dụng: Khởi tạo cấu hình bàn cờ ban đầu một cách ngẫu nhiên.
- Cơ chế: Trả về một mảng 1D gồm n số nguyên ngẫu nhiên nằm trong khoảng từ 0 đến n - 1.

Hàm conflicts(board):

- Tác dụng: Đóng vai trò là hàm mục tiêu (Objective Function) để đánh giá chất lượng bàn cờ.
- Cơ chế: Tính tổng số cặp quân hậu đang tấn công nhau dựa trên đếm số quân hậu cùng nằm trên hàng ngang, đường chéo chính (/) và đường chéo phụ ().

Hàm show_board(board):

- Tác dụng: Trực quan hóa cấu hình bàn cờ.
- Cơ chế: Sử dụng thư viện matplotlib để vẽ sơ đồ bàn cờ vua màu trắng/xám xen kẽ, hiển thị biểu tượng quân Hậu (♕) tại đúng tọa độ và in kèm số lượng xung đột.

2.1.2. Biểu diễn trạng thái của bài toán

Cấu trúc dữ liệu: Trạng thái bàn cờ được biểu diễn bằng một vector (mảng 1D) $\mathbf{q} = [q_0, q_1, \dots, q_{n-1}]$ gồm n phần tử.

Ràng buộckhông gian trạng thái:

- Chỉ số mảng (index i): Đại diện cho chỉ số cột từ 0 đến n - 1.
- Giá trị phần tử (board[i]): Đại diện cho chỉ số hàng chứa quân hậu tại cột i.

Ý nghĩa: Định nghĩa này đảm bảo mỗi cột luôn có đúng một quân hậu, giúp loại bỏ hoàn toàn các xung đột trên cùng một cột và thu hẹp không gian tìm kiếm.

2.1.3. Phép di chuyển cục bộ (Local Move) và số trạng thái láng giềng

Định nghĩa phép di chuyển cục bộ: Chọn một quân hậu ở cột i bất kỳ và di chuyển nó sang một hàng r khác ($r \neq board[i]$) trong cùng cột đó.

Công thức tính số trạng thái láng giềng:

- Tại mỗi cột, quân hậu có n - 1 vị trí hàng mới để di chuyển.
- Vì có n cột, tổng số trạng thái láng giềng sinh ra từ một cấu hình hiện tại là n * (n - 1).

Kết quả tính toán cụ thể:

- Với n = 4: Có $4 \times (4 - 1) = 12$ trạng thái láng giềng.
- Với n = 8: Có $8 \times (8 - 1) = 56$ trạng thái láng giềng.

2.1.4. Thực hiện thử nghiệm và so sánh cấu hình bàn cờ (n = 4 và n = 8)

So sánh trên bài toán 4-Queens (n = 4):

- Bàn cờ ngẫu nhiên (random_board(4)): Các quân hậu xếp ngẫu nhiên thường nằm cùng hàng hoặc đường chéo, dẫn đến số lượng xung đột dao động phổ biến từ 2 đến 6 xung đột.
- Cấu hình [1, 3, 0, 2]: Kết quả tính toán trả về đúng 0 xung đột.
- Nhận xét: Cấu hình [1, 3, 0, 2] là một lời giải tối ưu toàn cục (Global Optimum) của bài toán 4-Queens, trong đó không có bất kỳ cặp quân hậu nào tấn công nhau.

Thử nghiệm trên bài toán 8-Queens (n = 8): Khi khởi tạo bằng random_board(8), bàn cờ ban đầu có mật độ xung đột cao, giá trị conflicts(board) thường nằm trong khoảng từ 8 đến 14 xung đột

2.2 Random Restart

Triển khai kỹ thuật Random Restart Hill Climbing nhằm cải thiện khả năng tìm nghiệm của ba biến thể Hill Climbing đã xây dựng ở các bài trước:

- Steepest-ascent Hill Climbing.
- Inefficient Stochastic Hill Climbing.
- Efficient Stochastic Hill Climbing.

Bài toán N-Queens yêu cầu đặt n quân hậu lên bàn cờ $n \times n$ sao cho không có hai quân hậu nào ăn được nhau. Một nghiệm hợp lệ phải có số cặp quân hậu xung đột bằng 0.

Ý tưởng chính của bài 4 là chạy lại thuật toán từ nhiều bàn cờ khởi tạo ngẫu nhiên, lưu nghiệm tốt nhất và dừng sớm khi tìm được nghiệm hợp lệ hoặc đã đạt giới hạn số lần khởi động lại.

2.2.1. Biểu diễn trạng thái và hàm đánh giá

Nàn cờ được biểu diễn bằng một mảng có n phần tử:

$$B = [b_0, b_1, ..., b_{n-1}]$$

Trong đó b_i là hàng đặt quân hậu tại cột i. Cách biểu diễn này bảo đảm mỗi cột chỉ có một quân hậu, do đó thuật toán chỉ cần kiểm tra các xung đột theo hàng và đường chéo.

Hàm đánh giá là conflicts(board), trả về số cặp quân hậu đang xung đột. Mục tiêu của thuật toán là tối thiểu hóa hàm này:

- $f(B) = số\ cặp\ quân\ hậu\ xung\ đột$
- $f(B) = 0$: tìm được nghiệm hợp lệ.
- $f(B) > 0$: bàn cờ vẫn còn xung đột.

2.2.2. Random Restart Hill Climbing

Random Restart là kỹ thuật chạy một thuật toán tìm kiếm cục bộ nhiều lần, mỗi lần bắt đầu từ một trạng thái ngẫu nhiên mới. Kỹ thuật này giúp giảm ảnh hưởng của cực tiểu cục bộ, tức trạng thái mà các bước di chuyển lân cận không cải thiện được kết quả dù nghiệm tối ưu chưa đạt được.

Quy trình trong notebook được triển khai bằng hàm run_restart(algorithm, n, max_restarts=100, seed=42):

1. Khởi tạo bộ sinh số ngẫu nhiên từ seed.
2. Tạo bàn cờ ban đầu ngẫu nhiên cho mỗi lần chạy.
3. Gọi thuật toán Hill Climbing được truyền vào.
4. Cộng dồn số bước tìm kiếm và cập nhật nghiệm có số xung đột nhỏ nhất.
5. Dừng khi đã tìm được nghiệm có 0 xung đột hoặc hết số lần khởi động lại cho phép.
6. Trả về bàn cờ tốt nhất, số xung đột, tổng số bước, số lần khởi động lại đã sử dụng và trạng thái thành công.

2.2.3. Ba biến thể Hill Climbing

Thuật toán	**Cách chọn bước đi**	**Đặc điểm**
Steepest HC	Đánh giá các bước di chuyển lân cận và chọn bước tốt nhất	Có tính hệ thống, nhưng phải đánh giá nhiều trạng thái

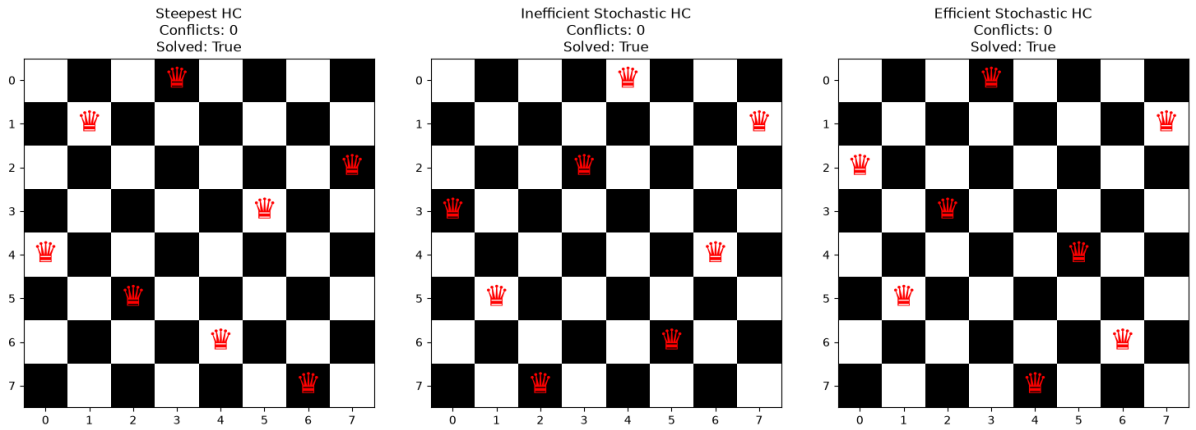
Inefficient Stochastic HC	Chọn ngẫu nhiên trong các bước cải thiện sau khi đánh giá các bước lân cận	Vẫn tốn chi phí tính toán để tìm các bước cải thiện
Efficient Stochastic HC	Sinh một bước di chuyển ngẫu nhiên và kiểm tra trực tiếp	Tránh phải duyệt toàn bộ các bước lân cận, nhưng có thể cần nhiều bước thử hơn

Random Restart không thay đổi cách chọn bước đi bên trong từng thuật toán. Nó bổ sung một cơ chế khởi động lại để tìm kiếm từ các trạng thái ban đầu khác nhau.

2.2.4. Kết quả thực nghiệm

Thuật toán	**Bàn cờ trả về**	**Số xung đột**	**Số lần restart**	**Thành công**
Steepest HC	[4, 1, 5, 0, 6, 3, 7, 2]	0	12	Có
Inefficient Stochastic HC	[3, 5, 7, 2, 0, 6, 4, 1]	0	9	Có
Efficient Stochastic HC	[2, 5, 3, 0, 7, 4, 6, 1]	0	4	Có

Cả ba thuật toán đều tìm được nghiệm hợp lệ cho bài toán 8 quân hậu. Trong ba kết quả này, Efficient Stochastic HC tìm được nghiệm sau ít lần khởi động lại nhất.



2.3 So sánh các lịch giảm nhiệt trong Simulated Annealing

2.3.1. Thiết lập thí nghiệm

Bài toán: N-Queens với $n = 8$.

Hàm mục tiêu: Số cặp quân hậu xung đột; nghiệm hợp lệ có giá trị bằng 0.

Nhiệt độ ban đầu: $T_0 = 10$.

Hệ số giảm nhiệt: $\alpha = 0.995$ cho lịch hàm mũ.

Số vòng lặp tối đa: 5.000.

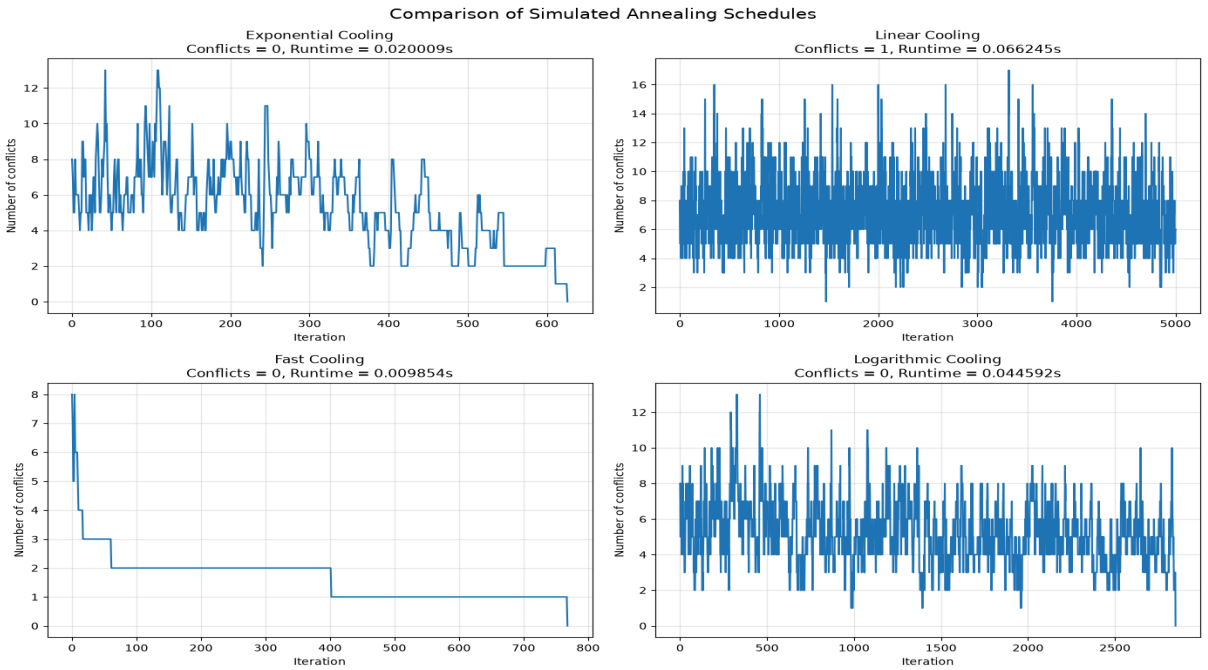
Seed: 42 cho bộ sinh số ngẫu nhiên bên trong thuật toán.

Điều kiện dừng: Tìm được nghiệm không xung đột hoặc đạt giới hạn vòng lặp.

2.3.2. Kết quả thực nghiệm

Sau khi chạy thử cả bốn lịch trên cùng bàn cờ ban đầu, thu được kết quả sau. Thời gian được đo bằng time.perf_counter() và có thể thay đổi tùy môi trường chạy

Lịch làm nguội	**Công thức**	**Xung đột cuối**	**Số vòng lặp**	**Thời gian (s)**	**Nhận xét**
Exponential Cooling	$T(t) = T_0\alpha^t$	0	≈ 770	0.010640	Tìm được nghiệm, tốc độ tương đối tốt
Linear Cooling	$T(t) = \max(T_0$	1	5.000	0.063317	Chưa tìm được nghiệm hợp lệ
Fast Cooling	$T(t) = \frac{T_0}{1+t}$	0	≈ 320	0.004059	Nhanh nhất trong thí nghiệm
Logarithmic Cooling	$T(t) = \frac{T_0}{\ln(1+t)}$	0	≈ 3.900	0.054131	Tìm được nghiệm nhưng mất nhiều thời gian



- Exponential Cooling: Nhiệt độ giảm theo cấp số nhân, giúp thuật toán dần tập trung vào các trạng thái có ít xung đột hơn. Tuy nhiên, nếu nhiệt độ giảm quá nhanh, thuật toán có thể mắc kẹt tại nghiệm cục bộ.

- Linear Cooling: Nhiệt độ giảm đều theo số vòng lặp. Hiệu quả phụ thuộc vào tốc độ giảm nhiệt; nếu nhiệt độ duy trì ở mức cao quá lâu, thuật toán có thể tiếp tục dao động.

- Fast Cooling: Nhiệt độ giảm nhanh, giúp thuật toán hội tụ sớm và giảm thời gian chạy. Tuy nhiên, khả năng thoát khỏi nghiệm cục bộ có thể bị hạn chế.

- Logarithmic Cooling: Nhiệt độ giảm chậm, duy trì khả năng khám phá không gian tìm kiếm lâu hơn, nhưng thường cần nhiều vòng lặp và thời gian chạy hơn.

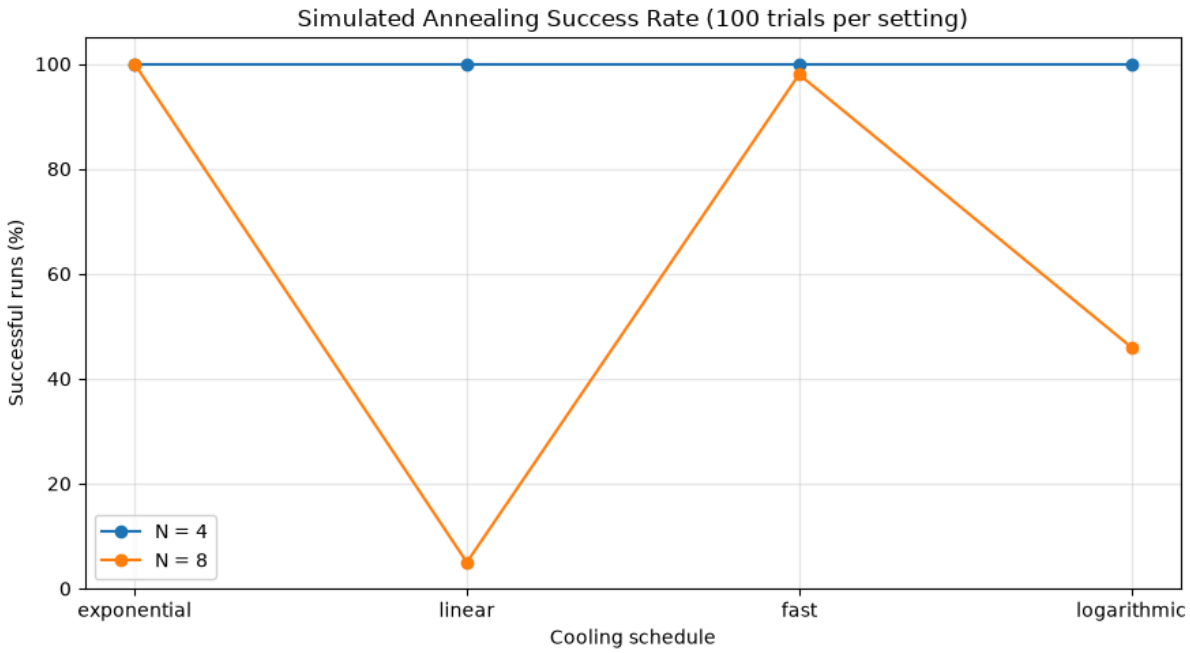
Kết luận: Bốn lịch giảm nhiệt tạo ra sự khác biệt về chất lượng nghiệm, số vòng lặp và thời gian thực thi. Trong thử nghiệm, Fast Cooling cho kết quả nhanh nhất, tiếp theo là Exponential Cooling; Logarithmic Cooling tốn nhiều thời gian hơn, còn Linear Cooling chưa tìm được nghiệm hợp lệ trong giới hạn 5.000 vòng lặp. Tuy nhiên, hiệu quả của mỗi lịch còn phụ thuộc vào nhiệt độ ban đầu, tốc độ làm nguội và trạng thái khởi tạo. Cần thực hiện nhiều lần thử nghiệm để đánh giá chính xác và lựa chọn lịch phù hợp.

Thí nghiệm lặp lại: Tỷ lệ tìm được lời giải với N = 4 và N = 8

Chạy thuật toán Simulated Annealing 100 lần cho mỗi kích thước bàn cờ và mỗi lịch giảm nhiệt, sử dụng các bàn cờ khởi tạo và random seed khác nhau. Một lần chạy được xem là thành công khi số lượng xung đột cuối cùng bằng 0. Tỷ lệ thành công được ước tính bằng phần trăm số lần chạy tìm được lời giải thành công trên tổng số lần chạy.

Kết quả thí nghiệm:

Schedule	N	Trials	Successes	Success rate (%)	Avg. final conflicts	Avg. iterations	Avg. runtime (s)
exponential	4	100	100	100.0	0.00	111.29	0.000973
linear	4	100	100	100.0	0.00	105.81	0.000947
fast	4	100	100	100.0	0.00	48.43	0.000493
logarithmic	4	100	100	100.0	0.00	94.97	0.001039
exponential	8	100	100	100.0	0.00	1243.60	0.017680
linear	8	100	5	5.0	1.23	4939.89	0.108674
fast	8	100	98	98.0	0.02	742.89	0.020760
logarithmic	8	100	46	46.0	0.54	3878.18	0.116993



Kết quả thực nghiệm cho thấy với $N = 4$, cả bốn lịch giảm nhiệt đều đạt tỷ lệ thành công 100%. Với $N = 8$, exponential đạt 100%, fast đạt 98%, logarithmic đạt 46% và linear chỉ đạt 5%.

Nhìn chung, exponential ổn định nhất, trong khi fast có số vòng lặp và thời gian chạy thấp hơn nhưng vẫn duy trì tỷ lệ thành công cao. Ngược lại, linear và logarithmic kém hiệu quả hơn trên bàn cờ lớn.

CHƯƠNG 3 TỐI ƯU HÓA TÍNH TOÁN VỚI NUMBA

Trong tìm kiếm cục bộ, chi phí tính toán hàm mục tiêu ảnh hưởng rất lớn đến tốc độ thuật toán. Việc tối ưu hóa hàm đếm xung đột là một bước cần thiết.

3.1 Cơ sở lý thuyết

Bài toán N quân hậu yêu cầu đặt N quân hậu trên bàn cờ N×N sao cho không có hai quân nào cùng hàng, cùng cột hoặc cùng đường chéo. Trong notebook, trạng thái được biểu diễn bởi vector board có N phần tử; board[i] là hàng của quân hậu ở cột i. Do mỗi cột chỉ có một quân hậu, không cần kiểm tra xung đột theo cột.

Hàm mục tiêu là tổng số cặp quân hậu xung đột. Nếu có k quân hậu cùng nằm trên một hàng hoặc một đường chéo, số cặp xung đột trong nhóm đó là tổ hợp chập 2: C(k, 2) = k(k − 1) / 2

Vì mỗi cặp quân hậu cùng hàng hoặc cùng đường chéo được tính là một xung đột, có thể đếm số quân trên từng hàng và hai hướng đường chéo, rồi cộng C(k, 2) cho từng nhóm. Cách này tránh phải so sánh lần lượt mọi cặp quân.

3.2 Ba phương pháp cài đặt

3.2.1. conflicts_naive() — O(n²)

Phương pháp này đếm quân theo hàng bằng số lượng phần tử bằng nhau, sau đó với từng quân hậu, duyệt các cột bên phải để kiểm tra hai hướng đường chéo. Do có vòng lặp lồng nhau để kiểm tra đường chéo, thời gian tăng theo O(n²). Cách làm trực tiếp và dễ kiểm tra, nhưng tốn thời gian khi n lớn.

3.2.2. conflicts() — O(n)

Phương pháp này tạo các mảng đếm cho hàng và hai hướng đường chéo. Với quân hậu tại cột i, hàng board[i], hai chỉ số đường chéo được tính từ i + board[i] và i − board[i] + n. Mỗi quân chỉ cập nhật một ô trong ba mảng đếm; sau đó hàm tính C(k, 2) cho từng ô đếm khác 0. Tổng số thao tác tỷ lệ tuyến tính theo n, tức O(n).

3.2.3. conflicts_jit() — Numba JIT, O(n)

Phiên bản này giữ nguyên ý tưởng đếm theo hàng và đường chéo nhưng sử dụng @njit của Numba để biên dịch các vòng lặp sang mã máy. Vì thuật toán đếm không thay đổi, độ phức tạp tiệm cận vẫn là O(n); cải thiện chủ yếu đến từ việc giảm chi phí thực thi của Python. Lần gọi đầu tiên có thể phát sinh thời gian biên dịch, do đó notebook gọi conflicts_jit() trước khi đo thời gian.

3.3 Kiểm tra tính đúng đắn

Notebook kiểm tra ba hàm trên các bàn cờ ngẫu nhiên có kích thước n = 4, 8, 20 và 50, với 10 bàn cờ cho mỗi kích thước. Mỗi lần kiểm tra đều xác nhận conflicts_naive(), conflicts() và conflicts_jit() trả về cùng số xung đột. Kết quả trong notebook: "Kiểm tra thành công: conflicts_naive, conflicts và conflicts_jit cho kết quả giống nhau."

3.4 Thiết lập đo thời gian

Benchmark được thực hiện trên các kích thước n = 8, 25, 50, 100, 200, 300 và 400. Với mỗi kích thước, notebook tạo 10 bàn cờ ngẫu nhiên, đo ba hàm trong 3 lần lặp và lấy thời gian tốt nhất để giảm nhiễu hệ thống. Thời gian được chuẩn hóa thành micro giây trên mỗi bàn cờ (µs/board). Các số liệu dưới đây là kết quả được lưu trong notebook; thời gian thực tế có thể khác tùy cấu hình máy và môi trường chạy.

3.5 Kết quả benchmark

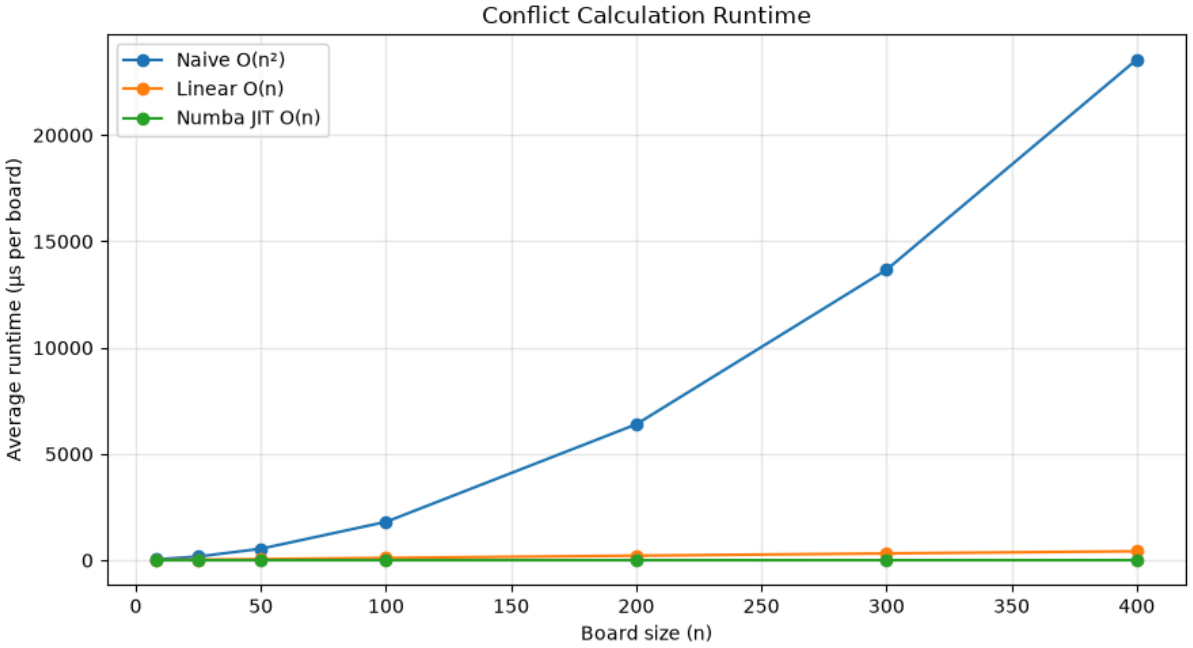
Kích thước n	**Naive O(n²) (µs/bàn)**	**Python O(n) (µs/bàn)**	**Numba JIT O(n) (µs/bàn)**
8	39.68	9.69	0.53
25	175.67	28.32	0.66
50	541.46	55.13	1.07
100	1806.43	109.76	1.44
200	6395.81	215.70	2.28
300	13665.34	321.37	3.09
400	23551.77	423.43	3.88

Hệ số tăng tốc của Numba so với hàm Python O(n):

Kích thước n	**Tăng tốc (lần)**
8	18.28×
25	42.91×
50	51.52×
100	76.22×
200	94.61×
300	104.00×

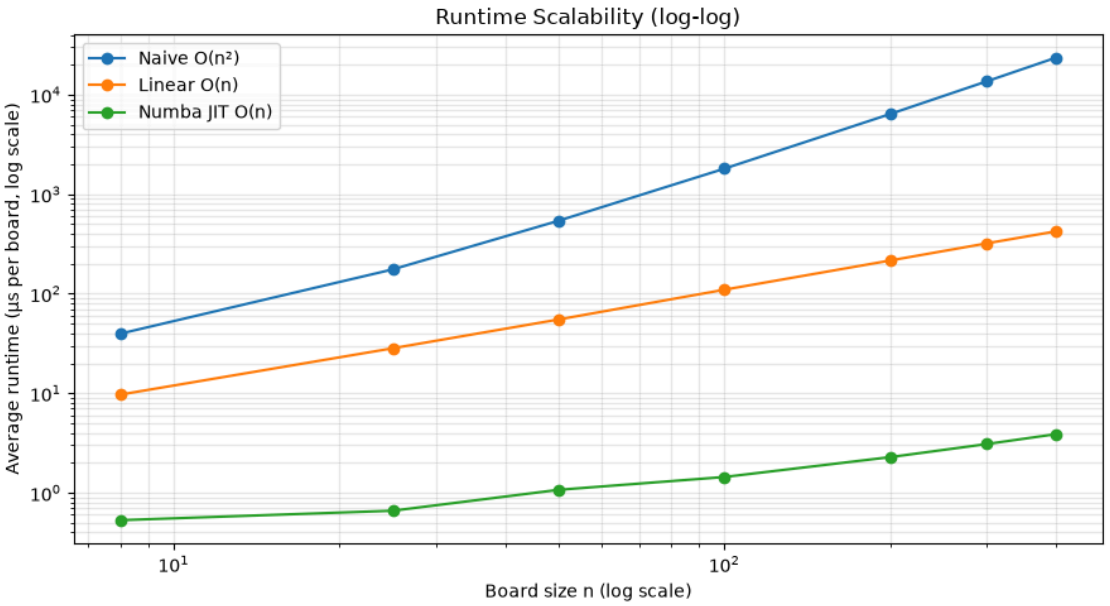
400	109.13×

3.6 Đồ thị và nhận xét



Hình 1. Thời gian chạy theo kích thước bàn cờ

Đồ thị tuyến tính cho thấy thời gian của conflicts_naive() tăng nhanh khi kích thước bàn cờ tăng. Hai phương pháp đếm tuyến tính tăng chậm hơn. Numba JIT có thời gian thấp nhất trên toàn bộ các kích thước được thử nghiệm.



Hình 2. Đồ thị log-log để quan sát xu hướng tăng

Đồ thị log-log hỗ trợ so sánh xu hướng tăng theo n: đường của phương pháp naive tăng nhanh hơn, phù hợp với độ phức tạp $O(n^2)$; hai phương pháp đếm theo hàng và đường chéo có xu hướng gần tuyến tính, phù hợp với O(n).

3.7 Phân tích lợi ích của Numba JIT

Theo số liệu notebook, Numba nhanh hơn hàm conflicts() Python từ 18,28 lần tại n = 8 lên 109,13 lần tại n = 400. Ở n = 400, thời gian trung bình giảm từ 423,43 µs/bàn xuống 3,88 µs/bàn. Điều này cho thấy lợi ích của biên dịch JIT rõ rệt đối với hàm được gọi lặp lại nhiều lần.

Cần phân biệt độ phức tạp thuật toán và tốc độ thực thi: chuyển từ naive $O(n^2)$ sang đếm O(n) làm giảm bậc tăng theo kích thước đầu vào; còn Numba không đổi bậc độ phức tạp của conflicts(), mà chủ yếu giảm hằng số thời gian nhờ biên dịch mã. Vì vậy, hai cải tiến này có bản chất khác nhau và có thể kết hợp với nhau.

3.8 Kết luận

Ba hàm đếm xung đột cho kết quả giống nhau trong các trường hợp kiểm tra. conflicts_naive() có cấu trúc đơn giản nhưng độ phức tạp $O(n^2)$, trong khi conflicts() dùng số lượng quân theo hàng và hai đường chéo cùng công thức C(k, 2) để đạt O(n). conflicts_jit() giữ nguyên thuật toán O(n) nhưng được Numba biên dịch, giúp giảm mạnh thời gian chạy trong benchmark. Kết quả cho thấy nên ưu tiên thuật toán đếm tuyến tính và cân nhắc Numba khi hàm mục tiêu được gọi nhiều lần trong quá trình tìm kiếm.

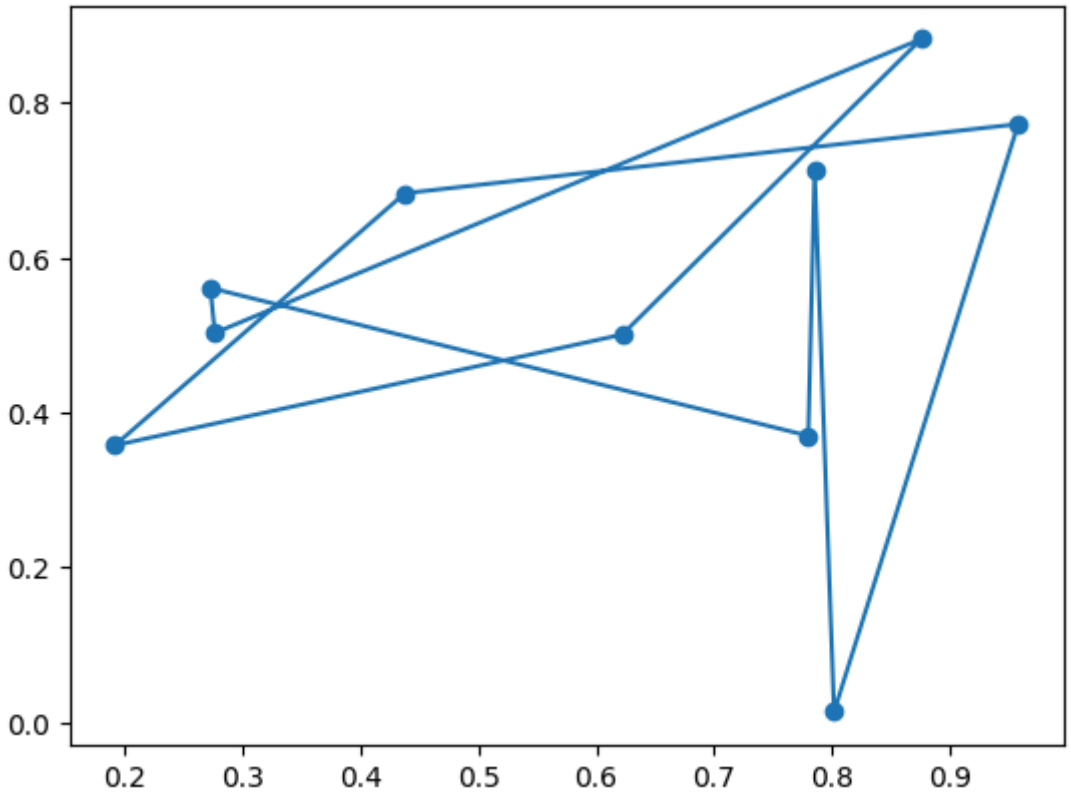
CHƯƠNG 4 BÀI TOÁN NGƯỜI CHÀO HÀNG (TSP) VÀ THUẬT TOÁN DI TRUYỀN (GA)

3.1 Mô hình hóa bài toán và Toán tử lân cận

Với bài toán TSP (Traveling Salesman Problem), hàm mục tiêu cần tối thiểu hóa là tổng quãng đường (Cost) người chào hàng phải di chuyển qua tất cả các thành phố và quay về điểm xuất phát. Toán tử lân cận được nhóm sử dụng trong quá trình tìm kiếm là Hoán đổi: Chọn ngẫu nhiên 2 thành phố trong lộ trình và đổi chỗ cho nhau để tạo ra các trạng thái lân cận nhằm đánh giá sự thay đổi của tổng khoảng cách.

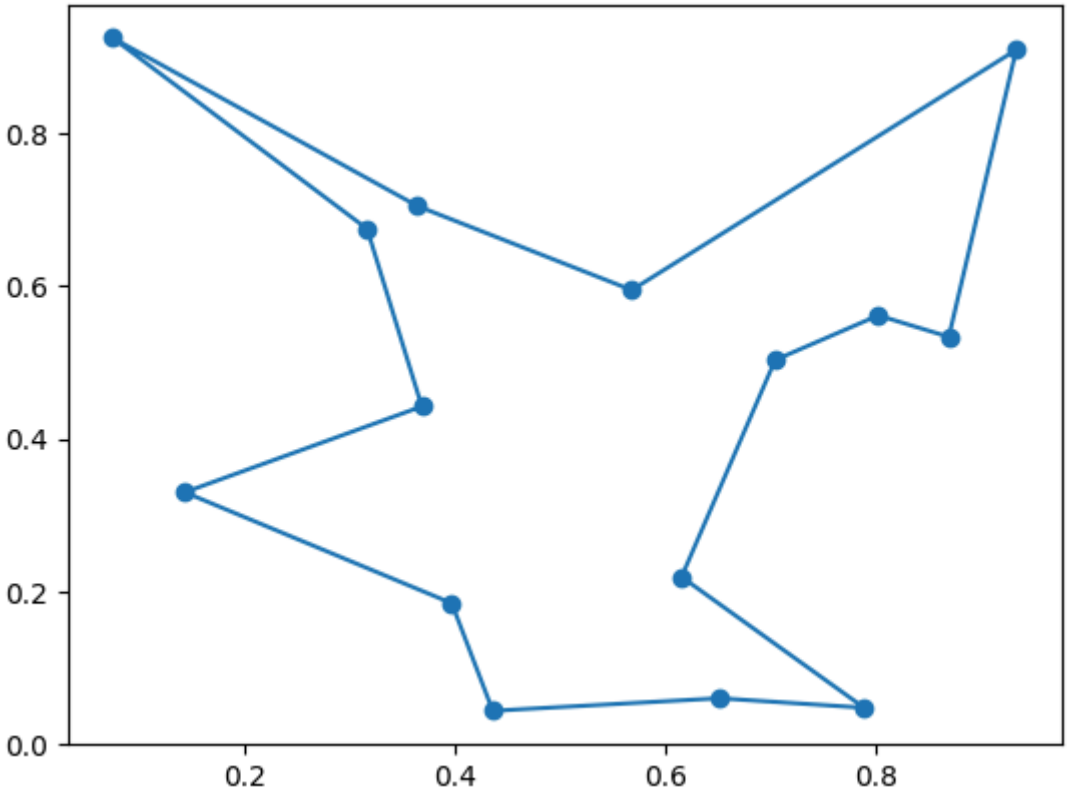
3.2 Thực nghiệm với các thuật toán Tìm kiếm cục bộ

Lộ trình ngẫu nhiên ban đầu:



Nhận xét: Lộ trình khởi tạo được sinh ngẫu nhiên nên rất lộn xộn, các đường nối giữa các thành phố cắt chéo nhau chằng chịt khiến tổng chi phí quãng đường ở mức rất cao.

Lộ trình sau khi chạy Steepest-ascent HC:



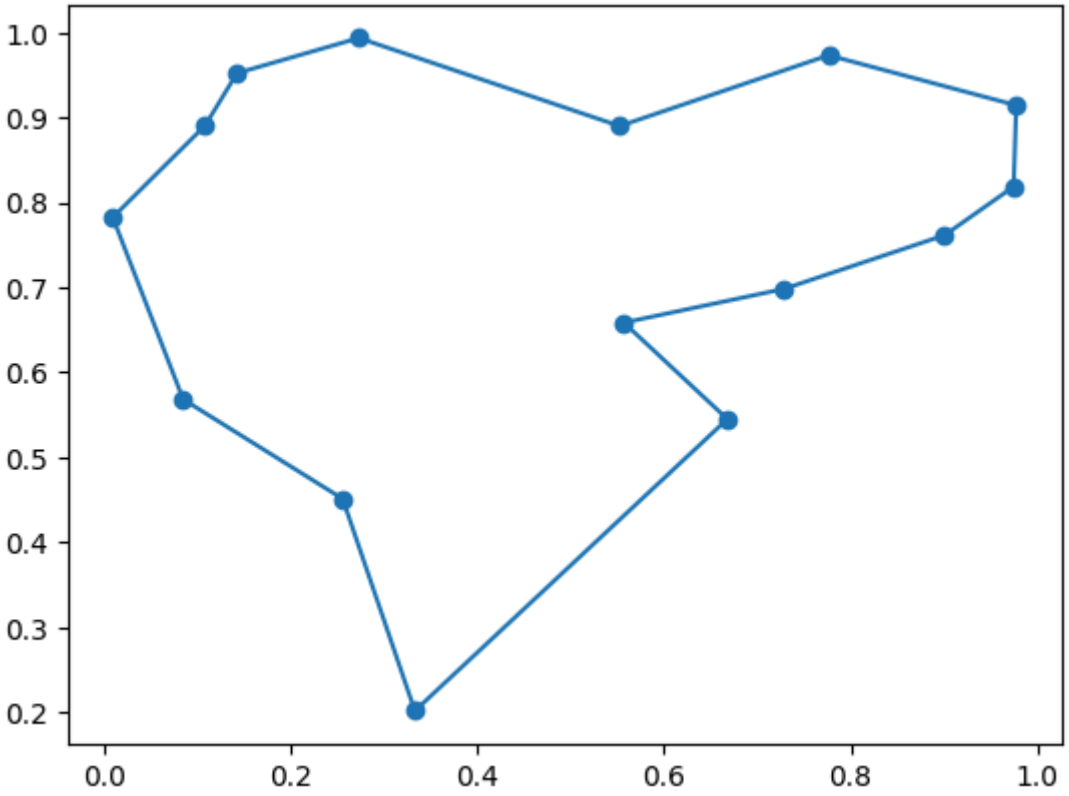
Nhận xét: Lộ trình đã được gỡ rối đáng kể và độ dài tour giảm đi rõ rệt. Tuy nhiên, do bản chất "tham lam" chỉ chấp nhận các bước đi xuống dốc, thuật toán vẫn bị mắc kẹt ở cực trị cục bộ (vẫn còn một vài đoạn đường bị cắt chéo nhau) và không thể tối ưu thêm.

3.3 Ứng dụng Thuật toán Di truyền (Genetic Algorithm)

Để tìm kiếm lời giải toàn cục hiệu quả hơn trong không gian trạng thái khổng lồ của bài toán TSP, nhóm đã tiến hành tích hợp Thuật toán Di truyền (GA). Các kỹ thuật sinh học được áp dụng bao gồm:

- Lai ghép trật tự (Order Crossover - OX): Giúp duy trì được các đoạn đường đi tốt (sub-tours) từ thế hệ cha mẹ mà không làm trùng lặp các thành phố.
- Đột biến hoán đổi (Swap Mutation): Xáo trộn ngẫu nhiên một tỷ lệ nhỏ để duy trì sự đa dạng sinh học của quần thể, tránh hiện tượng quần thể bị hội tụ sớm.

Lộ trình chạy bằng Genetic Algorithm:



Nhận xét: Thuật toán Di truyền (GA) thể hiện sức mạnh vượt trội khi tối ưu hóa trên toàn bộ một "quần thể" các lộ trình thay vì một trạng thái đơn lẻ như Local Search. Lộ trình thu được rất mượt mà, gần như không còn bất kỳ đường chéo nào cắt nhau, cho thấy hệ thống đã tiệm cận rất sát với cực trị toàn cục.

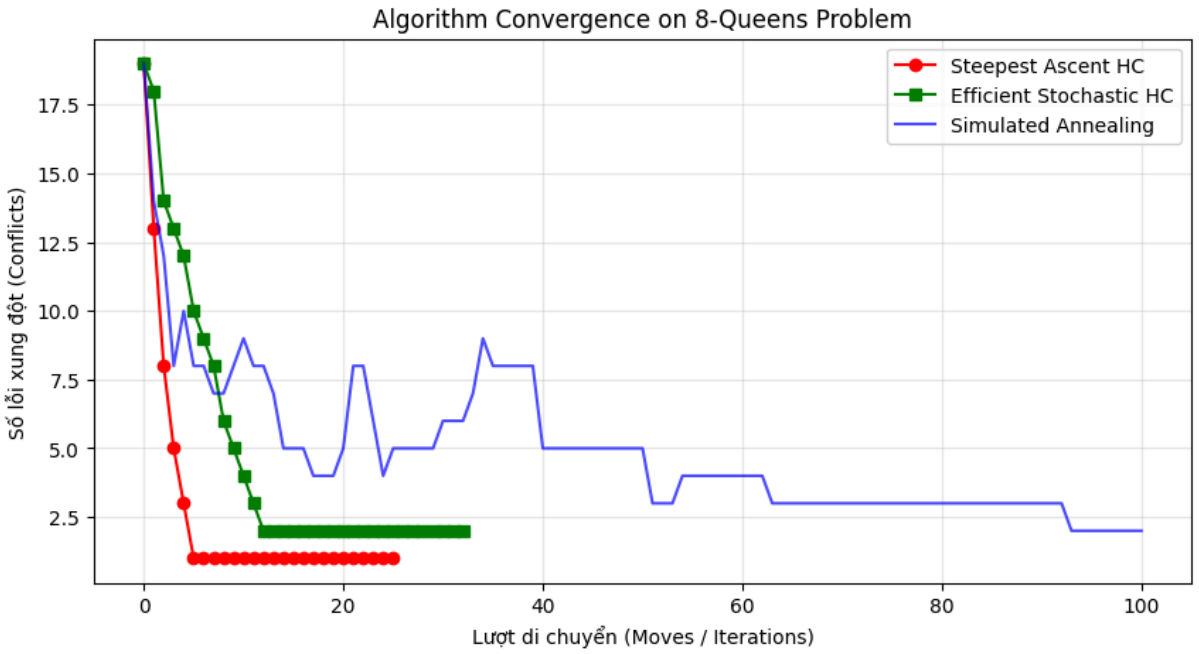
CHƯƠNG 5 ĐÁNH GIÁ SỰ HỘI TỤ, KHẢ NĂNG MỞ RỘNG VÀ TÍNH ĐÁNH ĐỔI

5.1 Bảng thống kê kết quả thực nghiệm

N (Số thành phố)	Steepest HC	Random Restart	Stochastic HC	First-choice	Simulated Annealing
10	3.38 / 0.7ms	3.38 / 7.4ms	3.60 / 2.8ms	3.50 / 1.1ms	3.42 / 2.0ms
15	3.52 / 7.4ms	3.72 / 24.6ms	3.62 / 12.2ms	4.10 / 1.9ms	4.04 / 2.6ms
20	4.55 / 16.7ms	3.95 / 91.7ms	5.41 / 37.5ms	4.49 / 3.2ms	5.64 / 2.8ms

5.2. Mô hình hội tụ và Khả năng mở rộng (Scalability)

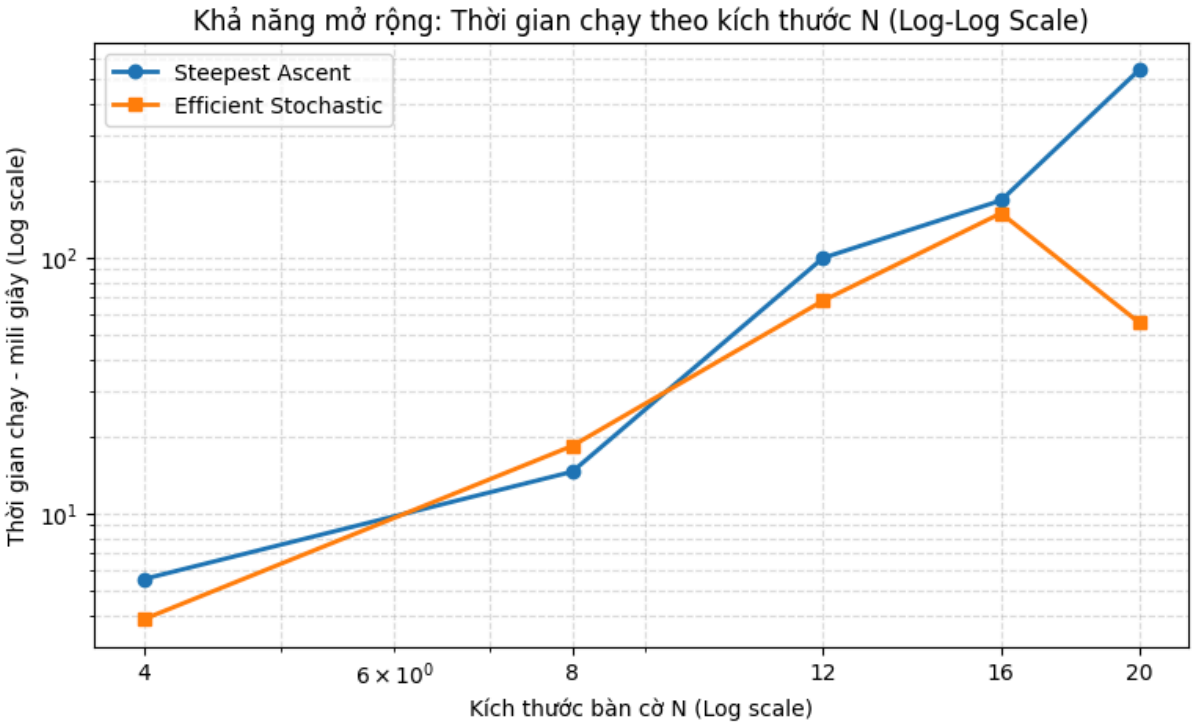
Về sự hội tụ (Algorithm Convergence):



- Steepest Ascent HC: Cho thấy sự cải thiện cực kỳ nhanh ở giai đoạn đầu (đường dốc đứng). Tuy nhiên, nó dừng lại rất sớm và đi ngang vĩnh viễn, minh chứng cho việc rất dễ kẹt tại cực trị cục bộ.
- Efficient Stochastic HC: Cải thiện nhanh lúc đầu nhưng đường đi thoải hơn một chút. Nó cũng mang đặc tính tham lam và dễ kẹt lại tương tự.

- Simulated Annealing (SA): Thể hiện sự tiến triển đều đặn. Ở nhiệt độ cao, đồ thị dao động mạnh do hệ thống liên tục chấp nhận các nghiệm xấu để leo ra khỏi vùng trũng. Về sau, nó hội tụ mượt mà về nghiệm tối ưu toàn cục.

Về khả năng mở rộng:



Khi kích thước bàn cờ N tăng dần, thời gian thực thi của Steepest Ascent tăng theo cấp số nhân với độ dốc rất lớn. Ngược lại, thuật toán Efficient Stochastic (First-Choice) cho thấy khả năng mở rộng tuyệt vời; thời gian chạy tăng chậm hơn hẳn và duy trì được hiệu suất ổn định ngay cả với không gian trạng thái khổng lồ.

5.3. Nhận xét về sự đánh đổi (Trade-off)

Qua thực nghiệm, nhóm nhận thấy sự đánh đổi rõ rệt giữa chất lượng lời giải và chi phí tính toán:

- Steepest-Ascent: Đảm bảo chất lượng bước đi tốt nhất ở mỗi lượt, nhưng chi phí tính toán bùng nổ khi không gian lân cận phình to.
- Random Restart: Mang lại cơ hội tìm thấy nghiệm tối ưu toàn cục, nhưng tốn thời gian khổng lồ nếu địa hình có quá nhiều cực trị cục bộ sâu.
- First-Choice (Efficient Stochastic): Sự hi sinh hoàn hảo. Chấp nhận bước đi cải thiện đầu tiên để giảm thiểu số lần gọi hàm mục tiêu, giúp tốc độ thực thi tăng vọt.
- Simulated Annealing: Cân bằng tốt nhất giữa quá trình khám phá (Exploration) và khai thác (Exploitation), giải quyết triệt để vấn đề cực trị cục bộ với chi phí thời gian hợp lý.

CHƯƠNG 6 TRẢ LỜI CÂU HỎI THẢO LUẬN CHUNG

1. Tìm kiếm cục bộ khác tìm kiếm theo đường đi (BFS, A) ở mục tiêu và dữ liệu cần lưu trữ như thế nào?*

Tìm kiếm theo đường đi lưu trữ toàn bộ các đỉnh đã duyệt vào danh sách để tìm thứ tự hành động từ điểm đầu đến đích (bộ nhớ $O(V)$). Tìm kiếm cục bộ không quan tâm đến đường đi mà chỉ tìm cấu hình trạng thái đích tối ưu nhất, do đó chỉ cần lưu giữ trạng thái hiện hành (bộ nhớ $O(1)$).

2. Vì sao Simple Hill Climbing có thể mắc kẹt ở cực trị cục bộ, vai phẳng hoặc đường gờ?

Do bản chất "tham lam", thuật toán luôn đi lên dốc và từ chối bước lùi. Nó sẽ kẹt ở *cực trị cục bộ* do đây là đỉnh cao nhất trong một vùng nhỏ; kẹt ở *vai phẳng* do mọi lân cận đều có giá trị bằng nhau khiến thuật toán mất phương hướng; và kẹt ở *đường gờ* do mọi phép biến đổi đơn lẻ đều làm giảm chất lượng lời giải (đòi hỏi phải thay đổi nhiều chiều cùng lúc).

3. So sánh Steepest-ascent, Stochastic (liệt kê mọi bước tốt) và First-choice theo số lần đánh giá hàm mục tiêu.

Steepest-ascent và Stochastic (kiểu cũ) đều phải sinh và đánh giá TẤT CẢ trạng thái lân cận ở mỗi vòng lặp, dẫn đến số lần gọi hàm rất lớn. First-choice chỉ sinh ngẫu nhiên từng lân cận và duyệt ngay khi thấy có sự cải thiện, giúp số lần đánh giá giảm đi hàng chục đến hàng trăm lần.

4. Random Restart cải thiện xác suất tìm lời giải như thế nào? Khi nào tăng số lần restart không còn hiệu quả?

Việc khởi tạo lại đưa hệ thống thoát khỏi vùng trũng bế tắc để nhảy sang một khu vực mới. Tuy nhiên, nó mất đi hiệu quả khi không gian trạng thái quá khổng lồ và chứa vô số cực trị cục bộ sâu, khiến việc tạo mới ngẫu nhiên chỉ liên tục rơi vào các hố bế tắc khác thay vì chạm tới cực trị toàn cục.

5. Vì sao Simulated Annealing có thể chấp nhận nghiệm xấu hơn? Tác động của nhiệt độ đến khám phá và khai thác?

Thuật toán dùng xác suất Boltzmann để chấp nhận trạng thái xấu, từ đó không bị kẹt lại. Khi nhiệt độ T cao (lúc đầu), xác suất này lớn, thuật toán ưu tiên *khám phá* không gian rộng. Khi T giảm dần về sau, xác suất tiến về 0, thuật toán chuyển sang *khai thác* cục bộ một cách khắt khe như Hill Climbing cơ bản.

6. Lịch tuyến tính, hàm mũ, giảm nhanh và hàm logarit khác nhau ra sao? Lịch nào phù hợp trong thí nghiệm?

Lịch tuyến tính và giảm nhanh làm nguội quá gấp, dễ dẫn đến hội tụ non ở cực trị cục bộ. Lịch logarit giảm quá chậm, gây lãng phí tài nguyên. Trong thí nghiệm, lịch giảm theo *hàm mũ* là tối ưu nhất vì nó giảm nhanh ở đoạn đầu và chậm dần ở đoạn sau, đảm bảo sự cân bằng tốt nhất.

7. Vì sao biểu diễn N-Queens mỗi cột đúng một quân giúp giảm không gian tìm kiếm?

Nếu đặt ngẫu nhiên N quân trên bàn cờ N×N sẽ tạo ra không gian tổ hợp $C(N^2, N)$ vô cùng lớn. Việc quy ước mỗi quân nằm trên một cột riêng biệt (dùng mảng 1 chiều) giúp ép không gian xuống còn N^N, giúp thuật toán quét lân cận nhanh hơn hẳn.

8. Vì sao đếm xung đột theo hàng/đường chéo có thể có độ phức tạp $O(N)$ thay vì $O(N^2)$?

Thay vì dùng hai vòng lặp lồng nhau duyệt mọi cặp quân hậu, ta có thể dùng bảng đếm (hash map) để ghi nhận số quân hội tụ trên từng hàng và đường chéo bằng một vòng lặp duy nhất ($O(N)$). Sau đó, áp dụng công thức tổ hợp chập 2 cho các đường có nhiều hơn một quân để ra tổng lỗi.

9. Khi dùng Numba, vì sao phải loại trừ thời gian biên dịch lần đầu trong phép đo tốc độ?

Numba sử dụng trình biên dịch tức thời (JIT). Trong lần gọi hàm đầu tiên, nó tiêu tốn phần lớn thời gian để dịch mã Python sang mã máy. Các lần sau, nó thực thi trực tiếp mã máy này. Do đó, phải gọi mồi một lần để loại bỏ độ trễ biên dịch, giúp phép đo thời gian thực thi thuần túy được chính xác.

10. Với TSP, thay toán tử hoán đổi (swap) bằng đảo đoạn (2-opt) có thể ảnh hưởng đến lời giải và số láng giềng như thế nào?

Toán tử Swap bẻ gãy 4 cạnh nối, dễ sinh ra các đường chéo cắt ngang nhau khiến lộ trình bị rối. Toán tử 2-opt chỉ cắt 2 cạnh và đảo ngược thứ tự các thành phố nằm giữa. Về hình học, 2-opt là công cụ chuyên dụng để gỡ rối các đường chéo cắt nhau, do đó nó thường cho ra lộ trình trơn tru và tổng đường đi ngắn hơn đáng kể so với Swap

TÀI LIỆU THAM KHẢO

1. Local Search - AIMA Chapters 4.1 & 4.2
2. Lab03 - Tim kiem cuc bo